

DETECTION ENGINEERING & ALERT TRIAGE

SIEM Detection Engineering, Attack Simulation, Tuning and SOC Alert Investigation

Final Project Report

Analyst: Randy Lomoko

27th August, 2026

Executive Summary

This project demonstrates an end-to-end security monitoring workflow using Wazuh as the SIEM. I designed and tested custom detections mapped to MITRE ATT&CK techniques, generated controlled security telemetry on Windows and Ubuntu endpoints, validated that the detections fired, investigated the resulting alerts, and assigned analyst dispositions.

The project was deliberately structured around a SOC workflow rather than rule creation alone. The work progressed from log-source validation, through detection engineering and attack simulation, to alert triage and investigation. The final stage produced a triage record and a completed Checkpoint 4 sign-off.

The Windows endpoint generated command, PowerShell and scheduled-task telemetry. The Ubuntu endpoint generated SSH authentication telemetry. A controlled SSH password-guessing simulation using Hydra successfully triggered the custom SSH detection. Windows testing also produced evidence for command-shell, account-creation and scheduled-task activity.

Project Deliverables

- Working custom detection rules loaded into Wazuh.
- Controlled attack/simulation activity used to validate detections.
- Tuning and validation of the detection logic.
- SOC-style triage playbook and alert investigation process.
- Completed triage log and analyst dispositions.
- Written report covering rule design, simulation, tuning and triage.
- End-to-end presentation plan for one detection rule.

Scope and Objectives

The report documents the activities actually performed in the lab. The project objectives were to:

- Validate that Windows and Ubuntu endpoints were successfully forwarding security telemetry to Wazuh.
- Design custom detections around relevant MITRE ATT&CK techniques.
- Load and validate the detections in the Wazuh SIEM.
- Generate controlled activity to exercise the detections.
- Confirm that at least the required detection scenarios generated usable alerts.
- Investigate alerts using the fields and evidence available in Wazuh.
- Assign analyst dispositions and document the reasoning behind each decision.
- Compile the final triage log and complete the Phase 4 checkpoint.
- Produce a concise technical report suitable for instructor assessment.

Lab Architecture

Source	Telemetry	Destination
Windows Endpoint	Windows/Sysmon security telemetry	Wazuh Manager
Ubuntu Endpoint	SSH/system authentication telemetry	Wazuh Manager
Testing workstation	Controlled simulation activity	Windows / Ubuntu endpoints

Methodology

I used an iterative detection-engineering workflow. Each detection began with a security behavior or ATT&CK technique, was translated into observable telemetry, implemented in Wazuh, exercised in the lab, and then reviewed from an analyst perspective.

1. Identify the security behavior and relevant ATT&CK technique.
2. Determine what endpoint telemetry can expose the behavior.
3. Create a Wazuh detection condition using the available event fields.
4. Generate controlled activity in the lab.
5. Verify that the expected telemetry reaches Wazuh.
6. Confirm that the custom rule fires.
7. Review the alert for evidence, context and potential false positives.
8. Record the result in the final triage log.

Detection Engineering Principles

- Prefer observable behavior over a single static indicator.
- Use endpoint context such as command line, process image, user and source IP where available.
- Keep the detection understandable enough for an analyst to investigate.
- Treat a detection firing as an investigation trigger, not automatic proof of compromise.
- Use controlled simulations to validate that the rule is operational.
- Document limitations and potential false positives.

Phase 1: Log Source Validation

Phase 1 established the telemetry foundation for the project. I verified that both the Windows and Ubuntu agents were connected to Wazuh and that security-relevant events were visible in the SIEM.

Windows Endpoint Validation

The Windows endpoint successfully generated process and command-line telemetry. The Wazuh Discover view showed fields including the agent name, agent IP, rule ID, rule description, Windows command line and user.

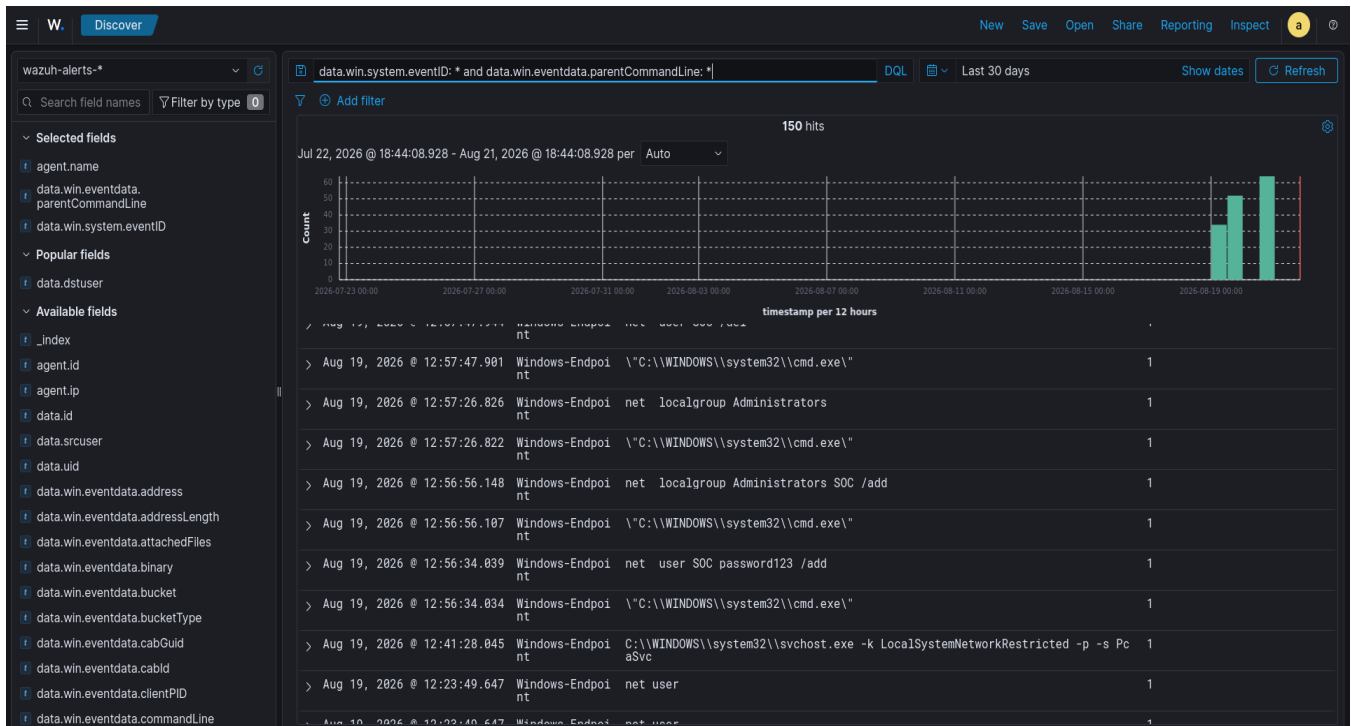


Figure 1: Windows Endpoint with Log Source Validation

Ubuntu Endpoint Validation

The Ubuntu endpoint successfully generated SSH authentication events. Wazuh displayed SSH success and failure events, source information and the affected account.

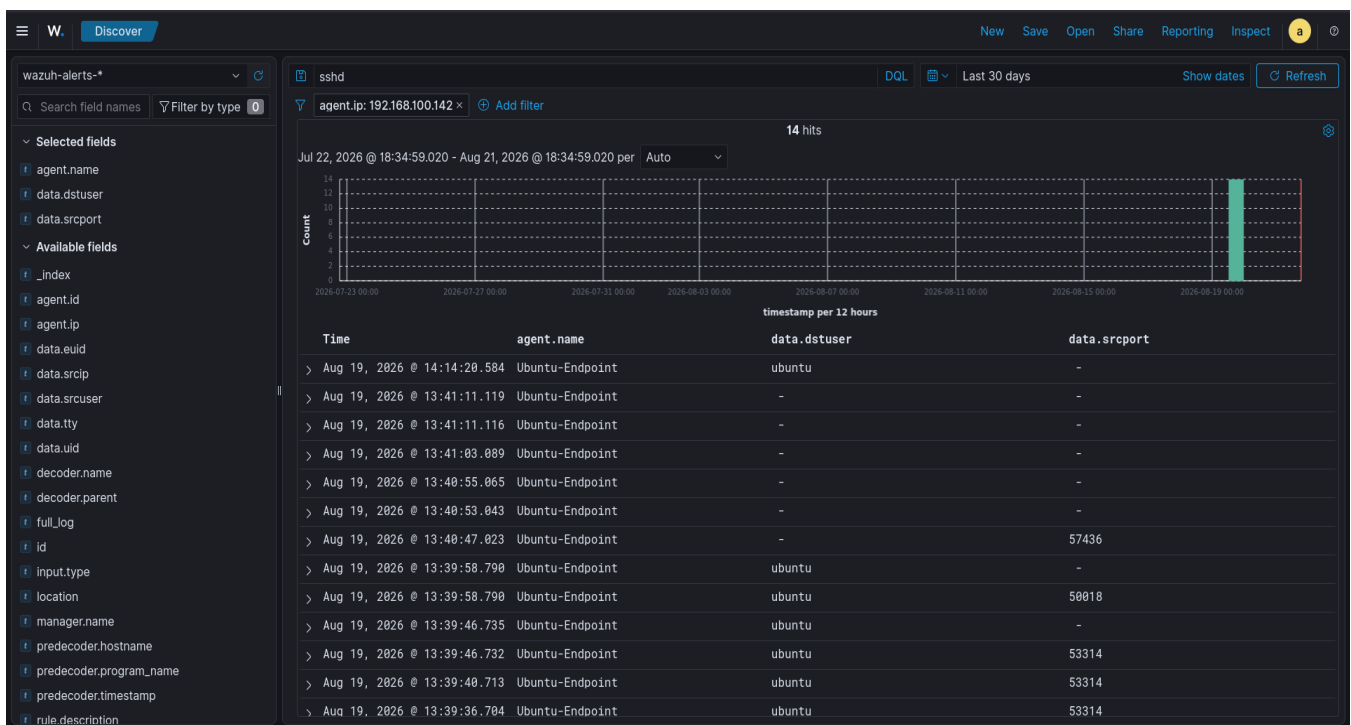


Figure 2: Ubuntu Endpoint with SSH logs

The result of Phase 1 shows that, both endpoint log sources were operational and suitable for detection testing.

Phase 2: Detection Engineering

Phase 2 focused on creating custom detections that could identify behaviors relevant to the project. The rules were loaded into Wazuh and validated against endpoint telemetry.

Detection Coverage

Detection	Platform	ATT&CK Technique	Purpose
Suspicious PowerShell	Windows	T1059.001 PowerShell	Identify suspicious PowerShell execution.
Encoded PowerShell	Windows	T1059.001 PowerShell	Identify PowerShell activity using encoded/obfuscated command content.
Windows Command Shell	Windows	T1059.003 Windows Command Shell	Identify suspicious cmd.exe command execution.
Scheduled Task	Windows	T1053.005 Scheduled Task/Job: Scheduled Task	Identify scheduled-task creation or modification.
SSH Password Guessing	Ubuntu	T1110.001 Password Guessing	Identify repeated SSH authentication failures from the same source.

Detection Logic

The detections were based on fields exposed by Wazuh. Windows rules relied heavily on process and command-line telemetry, while the SSH detection relied on repeated authentication-failure events and source-IP correlation.

- Windows Command Shell: detect command execution associated with cmd.exe and suspicious command-line behavior.
- PowerShell: detect PowerShell process execution and suspicious command content.
- Scheduled Task: detect use of schtasks.exe for task creation or modification.
- SSH Password Guessing: correlate repeated failed SSH authentication attempts from the same source within a defined time window.

Rule Quality Considerations

- A rule should produce evidence that an analyst can investigate.
- Rules should avoid relying on a single benign keyword where possible.
- Threshold-based detections must account for legitimate repeated failures.
- Command-line detections can be bypassed through alternative execution methods or renamed binaries.
- The quality of the underlying endpoint telemetry directly affects detection quality.

```

< local_rules.xml
30
31     <description>Suspicious PowerShell execution detected on $(win.system.computer).</description>
32
33     <mitre>
34         <id>T1059.001</id>
35     </mitre>
36
37     <group>attack,execution,powershell,</group>
38 </rule>
39
40 </group>
41
42
43 <group name="windows,sysmon,powershell,encoded_command,detection_engineering,">
44
45     <rule id="100101" level="12">
46         <if_group>sysmon_event1</if_group>
47
48         <field name="win.eventdata.image" type="pcre2">(?!\\(powershell|pwsh)\\.exe$</field>
49
50         <field name="win.eventdata.commandLine" type="pcre2">(?!^(?:[[:space:]])-(enc|encodedcommand)([[:space:]]|$)</field>
51
52         <description>Encoded PowerShell command detected on $(win.system.computer).</description>
53
54         <mitre>
55             <id>T1059.001</id>
56         </mitre>
57
58         <group>attack,execution,powershell,encoded_command,</group>
59 </rule>
60
61 </group>

```

Figure 2: Wazuh rule configuration for Windows

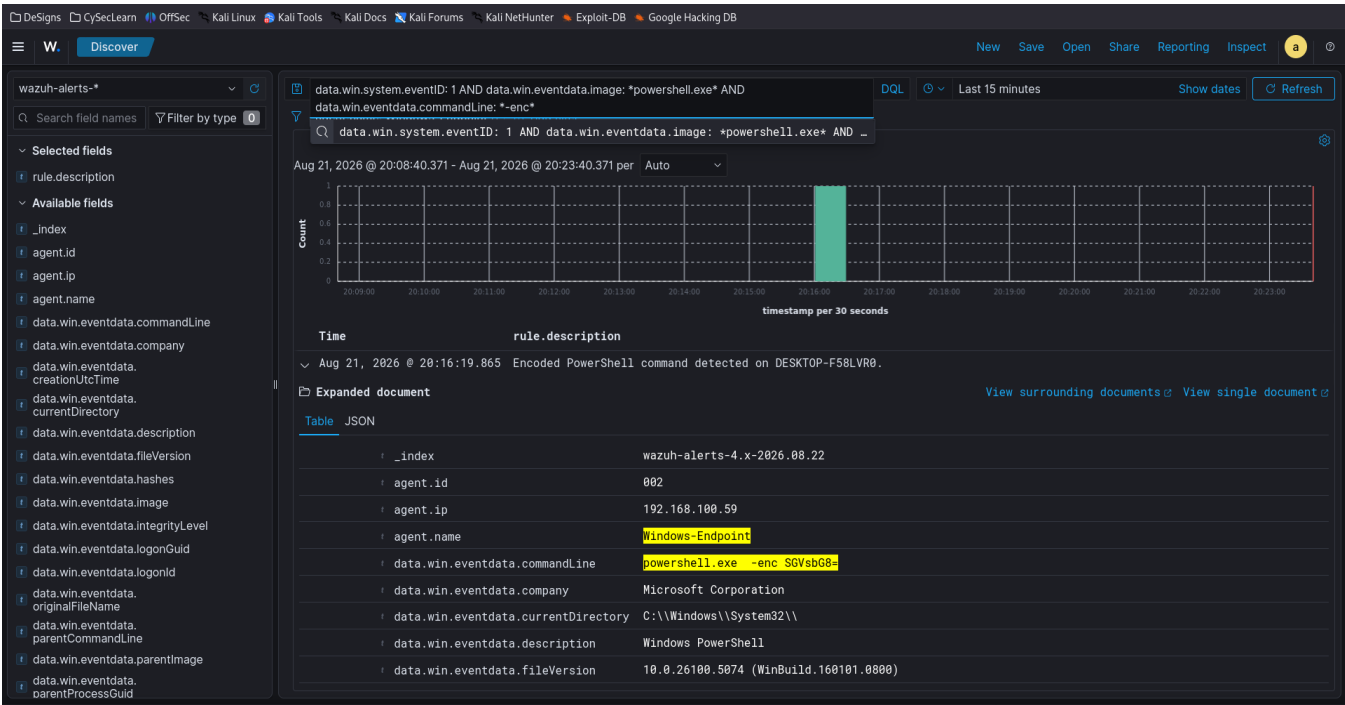


Figure 3: Windows Endpoint rule firing

Phase 3: Attack Simulation & Tuning

Phase 3 validated the detections by generating controlled activity on the lab endpoints. The goal was not to compromise a real system but to reproduce recognizable attack behaviors and verify telemetry.

Atomic Red Team Simulation

I simulated some of the MITRE ATT&CK techniques using Invoke-Atomic Red Team with the Windows endpoint. A controlled, repeatable test was conducted using Atomic Red Team as it will generate telemetry data that is similar to what an adversary would produce in a real attack, without compromising a system.

The simulations were mostly directed at the Phase 2 detections developed using the Windows platform, specifically:

Detection	MITRE ATT&CK Technique	Simulation Purpose
Suspicious PowerShell	T1059.001 – PowerShell	Generate PowerShell execution telemetry

Encoded PowerShell	T1059.001 – PowerShell	Test detection of encoded/suspicious PowerShell activity
Windows Command Shell	T1059.003 – Windows Command Shell	Generate cmd.exe execution activity
Scheduled Task	T1053.005 – Scheduled Task/Job: Scheduled Task	Generate scheduled-task activity

```

The system cannot find the file wordpad.exe.
Exit code: 0
Done executing test: T1059.003-4 Simulate BlackByte Ransomware Print Bombing
Executing test: T1059.003-5 Command Prompt read contents from CMD file and execute
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.
C:\Users\lucky\AppData\Local\Temp>@ECHO OFF
cmd.exe /c c:\windows\system32\calc.exe
More?
Exit code: 0
Done executing test: T1059.003-5 Command Prompt read contents from CMD file and execute
Executing test: T1059.003-6 Command prompt writing script to file then executes it
Exit code: 0
Done executing test: T1059.003-6 Command prompt writing script to file then executes it
PS C:\AtomicRedTeam>
PS C:\AtomicRedTeam>
ossec-agent

```

Figure 4: Invoke-Atomic Red Team tool in action

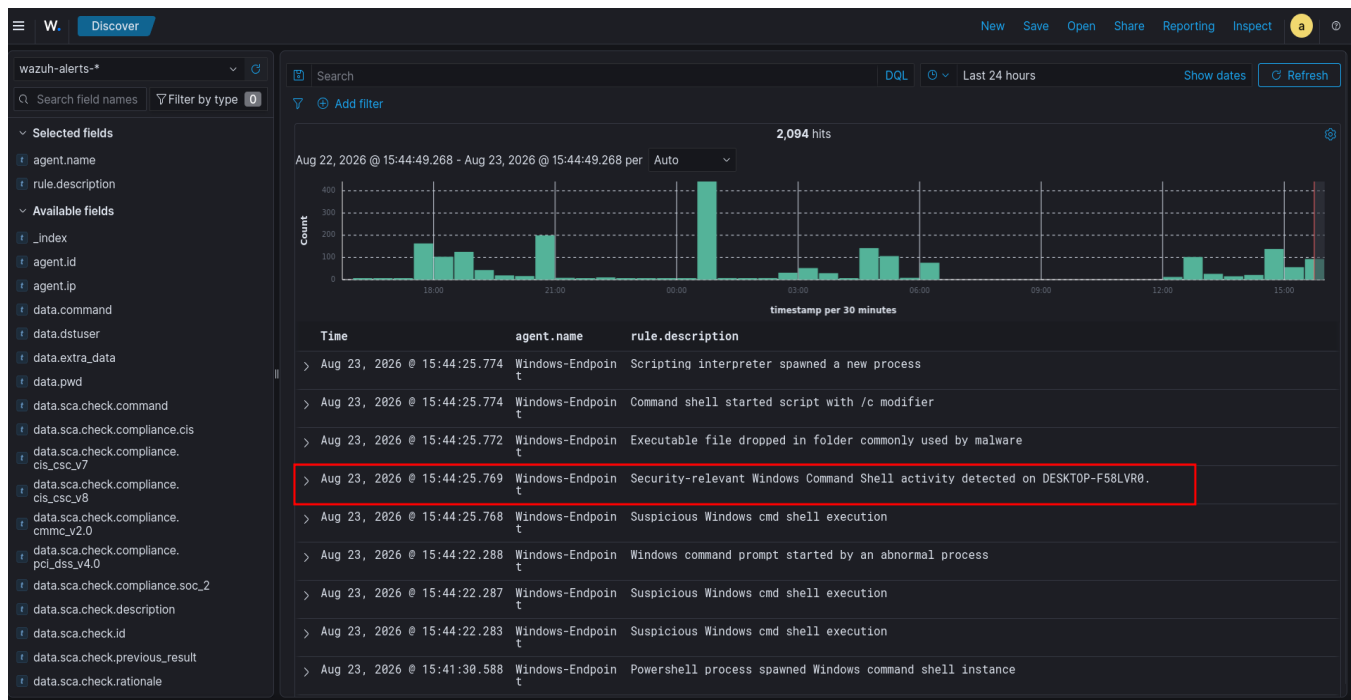


Figure 5: Atomic Red Team tool firing up rules

In the simulation phase, evidence was provided that the detection rules could be validated against controlled ATT&CK-like activity and that Wazuh was collecting endpoint telemetry data as a result of such activity. Testing set the groundwork for the next phase: investigating and triaging the alerts.

Phase 4: Alert Triage & Investigation

Triage Preparation

I established a repeatable workflow for handling alerts:

- Identify the alert and rule.
- Identify the affected endpoint and account.
- Review the command/process/network evidence.
- Establish what happened and whether the behavior is expected.
- Map the behavior to ATT&CK where applicable.
- Determine whether the alert represents a true or false positive.
- Record analyst reasoning and recommended response.

Triage Practice Batch

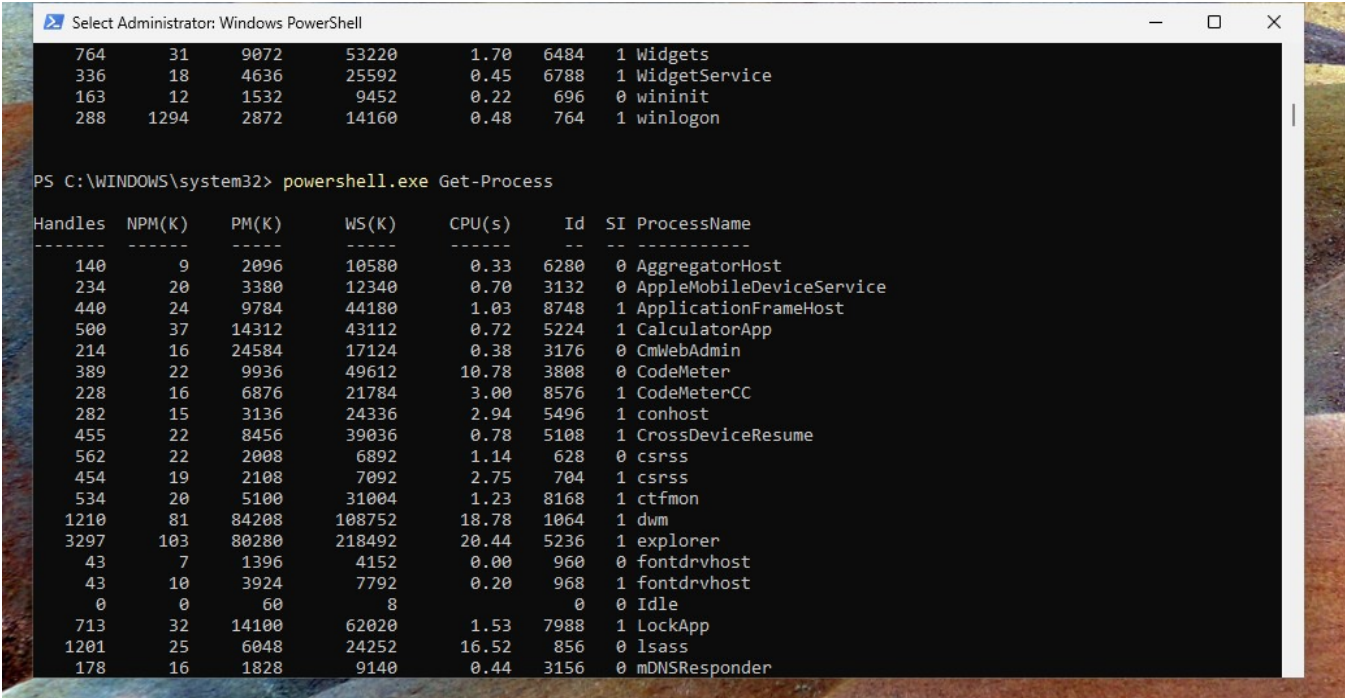


Figure 6: PowerShell Execution - False Positive

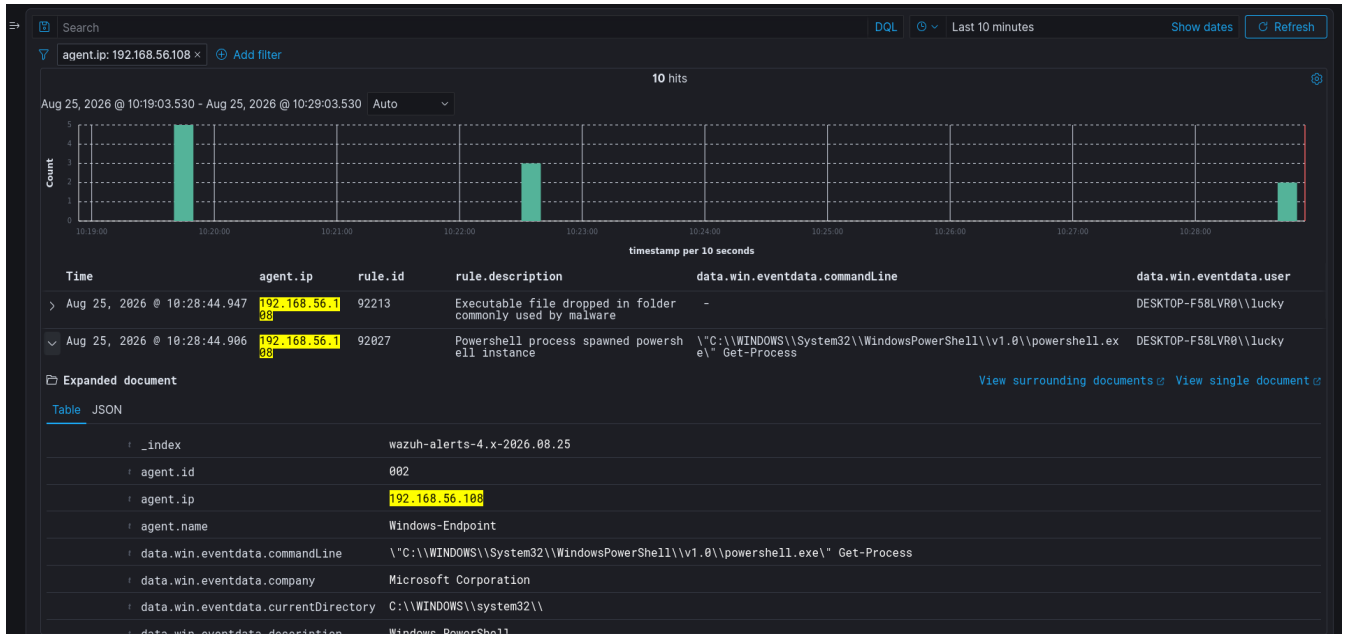


Figure 7: Wazuh alert showing False Positive

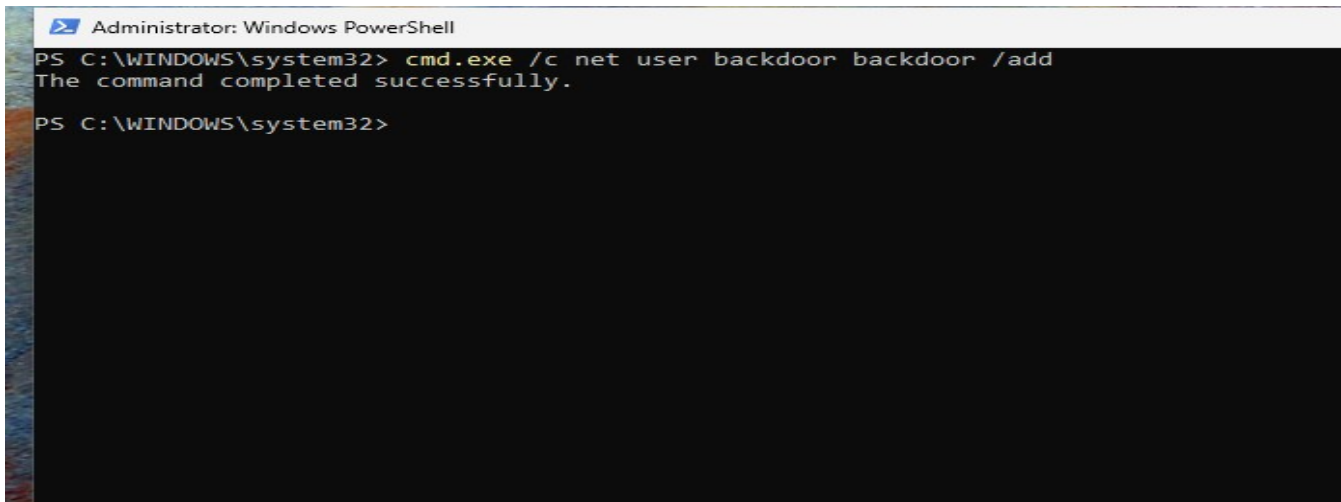


Figure 8: Windows Command shell - True Positive

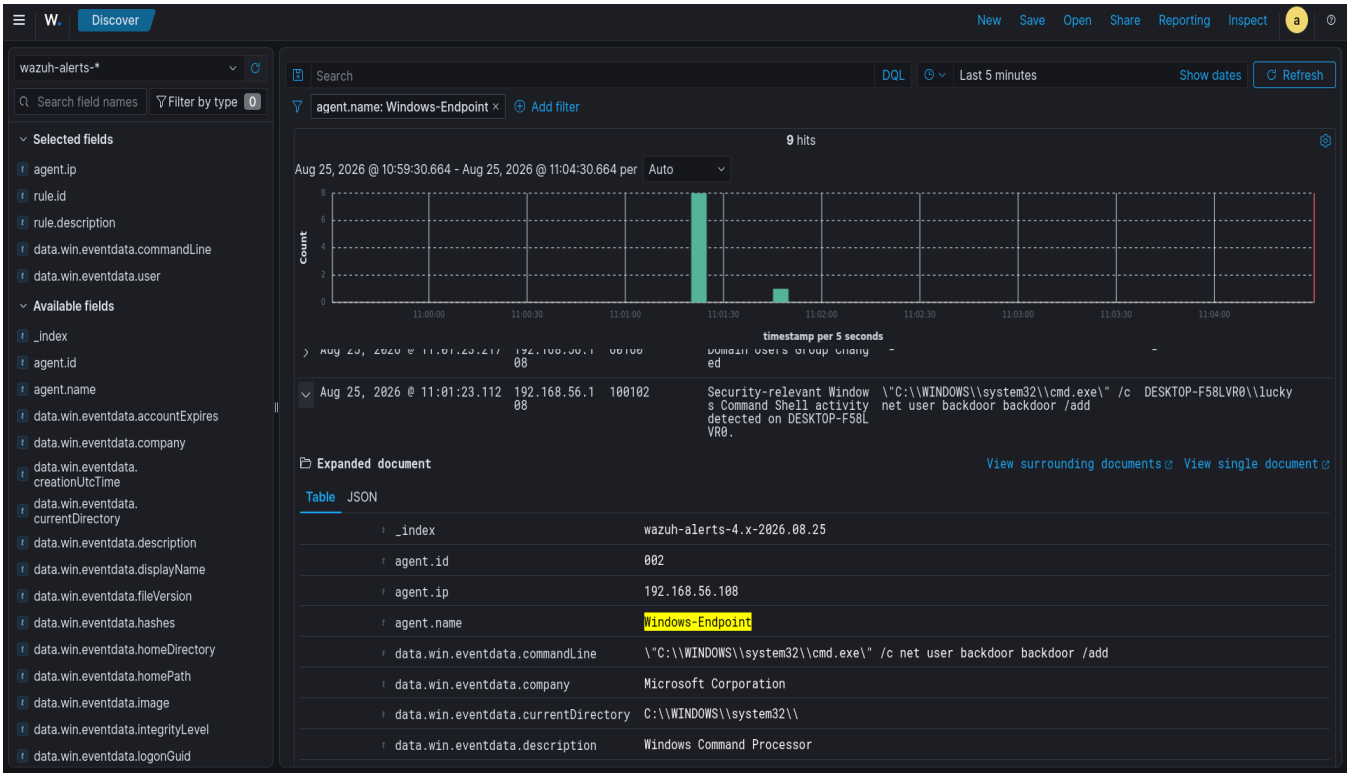


Figure 9: Detection rule fired True Positive alert

Individual Alert Investigation

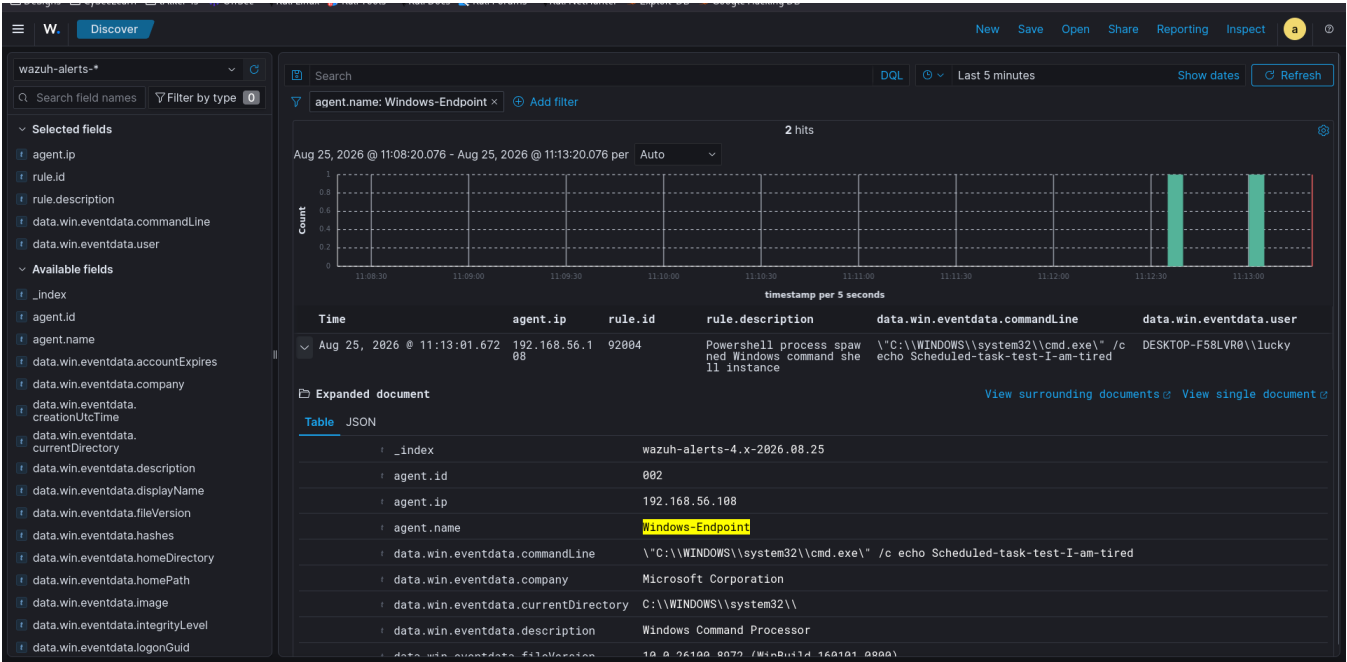


Figure 10: False Windows alert firing

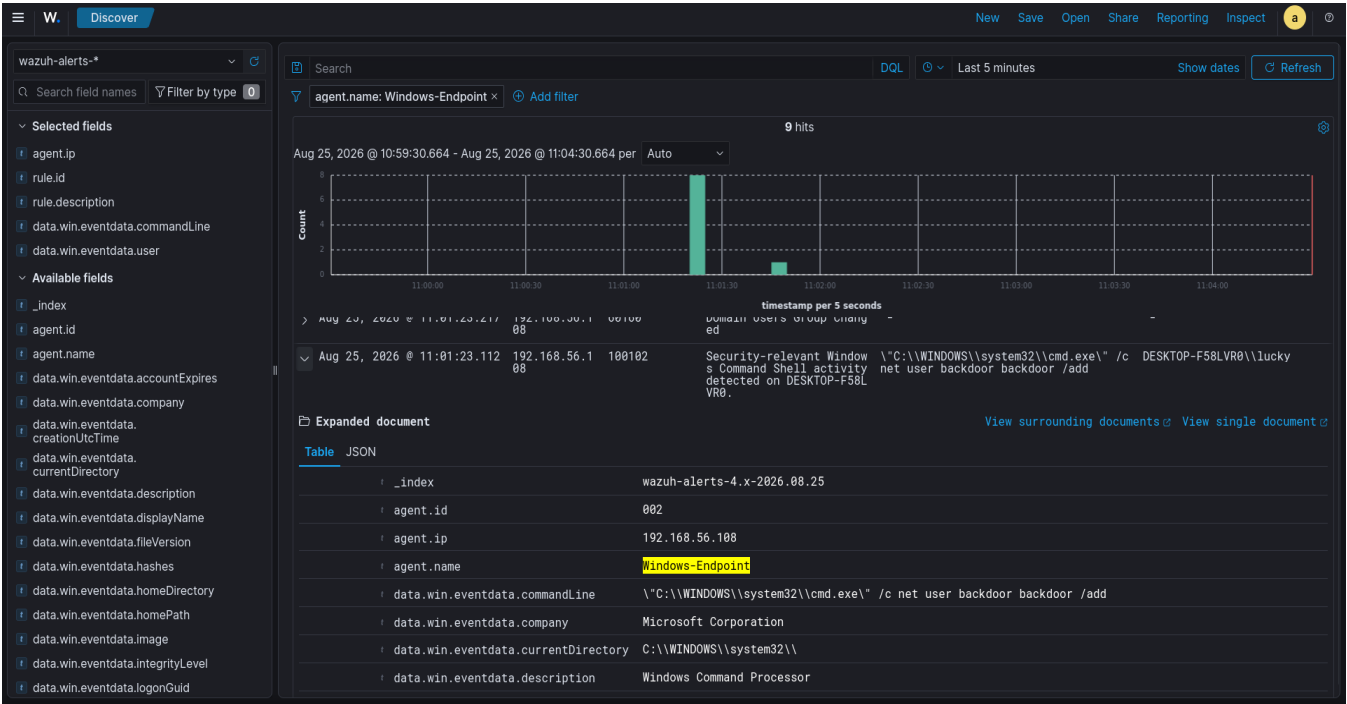


Figure 11: Malicious user creation - True Positive

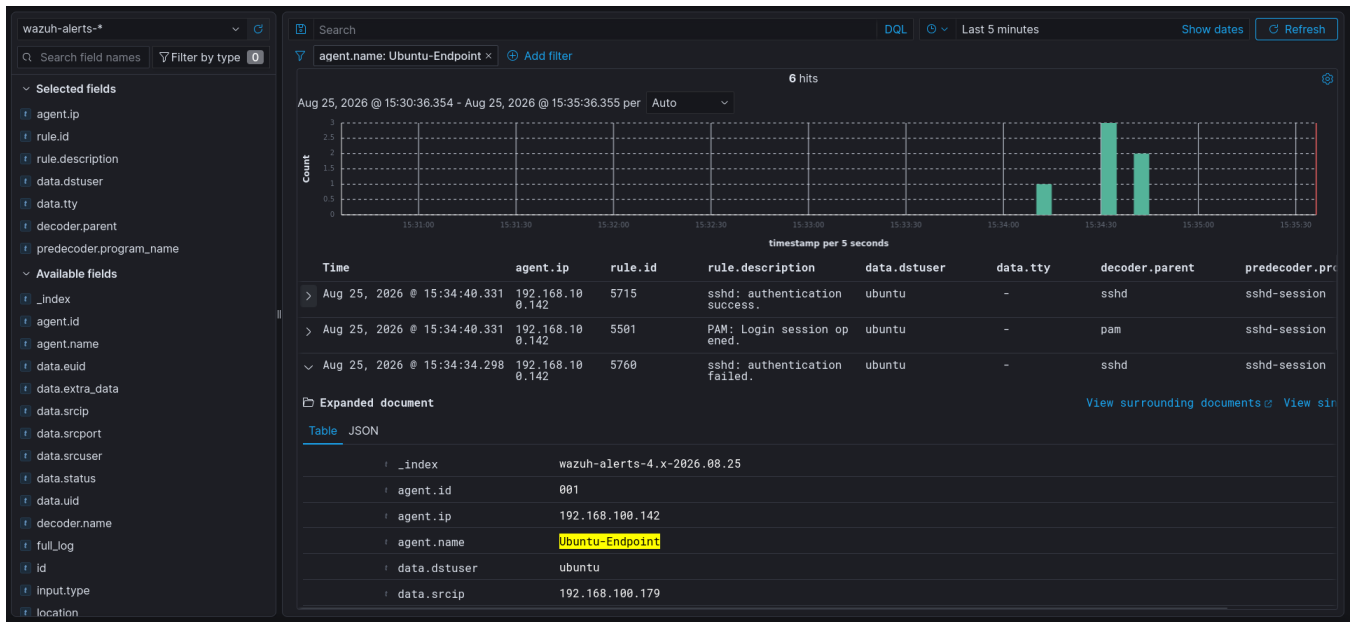


Figure 12: Normal SSH login with one failed attempt – False Positive

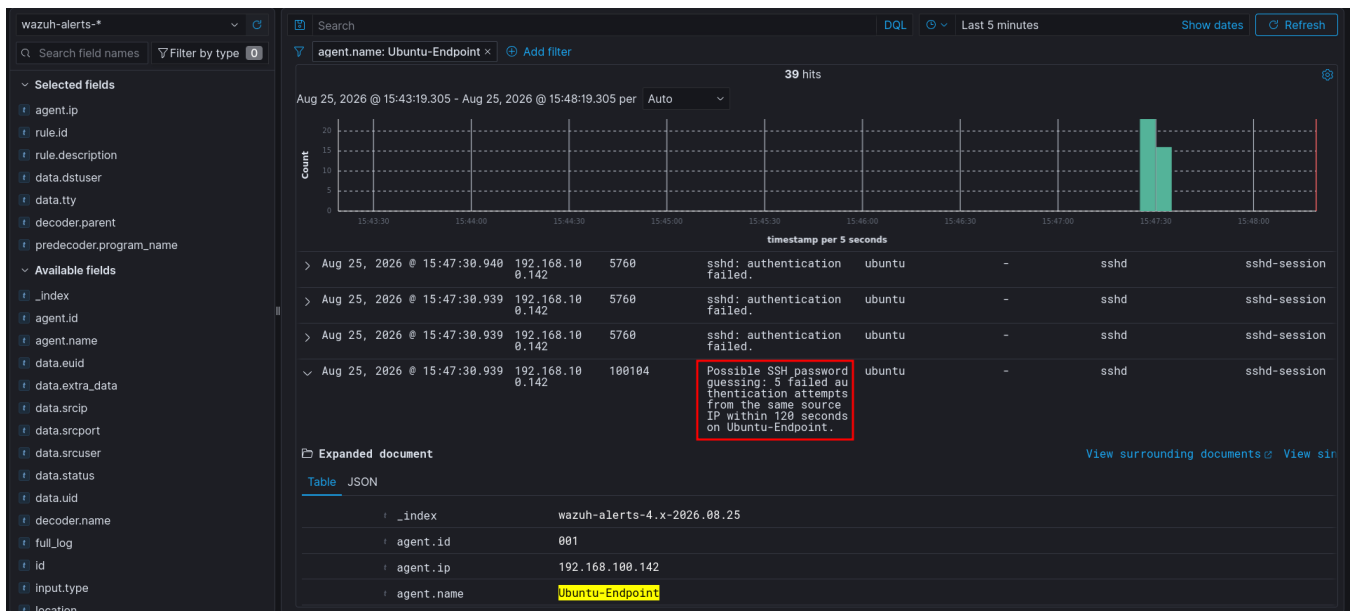


Figure 13: SSH brute force using hydra

Analyst Disposition & Reasoning

Alert / Activity	Disposition	Reasoning
Single failed SSH authentication	False Positive Simulated non-suspicious Activity	The event showed a single failed ssh login from a user. This can happen due to mistype of password.
Scheduled task creation	True Positive Simulated suspicious Activity	schtasks.exe created a scheduled task.
SSH password guessing	True Positive Simulated Malicious Activity	Repeated SSH authentication failures from one source within a short period matched the intended password-guessing simulation.
PowerShell / command-shell activity	False Positive Simulated a normal PowerShell or command-line Activity	The generated process and command telemetry from Wazuh normal rule does not matched the custom detection scenarios.

Triage Playbook

This triage playbook provides a consistent process for investigating security alerts generated by the Wazuh SIEM. It guides the analyst through initial validation, evidence collection, severity assessment, disposition, and escalation or closure.

The objective is to ensure that alerts are handled consistently and that every disposition is supported by observable evidence.

This playbook applies to the custom detections developed for this project:

Detection	MITRE ATT&CK	Endpoint
Suspicious PowerShell	T1059.001	Windows
Encoded PowerShell	T1059.001	Windows
Windows Command Shell	T1059.003	Windows
Scheduled Task	T1053.005	Windows
SSH Password Guessing	T1110.001	Ubuntu

Triage Workflow

- 1. Identify:** Record timestamp, host, user, rule ID, rule description, and source IP where applicable.
- 2. Validate:** Confirm that the alert corresponds to the activity described by the detection rule.
- 3. Investigate:** Review command line, process, parent process, user, network activity, timeline, and related alerts.
- 4. Establish Context:** Determine whether the activity is expected for the user, host, and environment.
- 5. Assign Severity:** Use the Low, Medium, High, or Critical definitions in this playbook.
- 6. Determine Disposition:** Classify the alert as malicious, suspicious, benign, false positive, or requiring further investigation.
- 7. Escalate or Close:** Escalate when risk or evidence warrants it; otherwise close with a documented reason.
- 8. Record Findings:** Document evidence, reasoning, disposition, and action taken.

Initial Alert Information

At minimum, record the following information when an alert is received:

Field	Purpose
Timestamp	When the activity occurred
Host / Agent	Which endpoint generated the alert
Rule ID	Which detection triggered
Rule Description	What behavior was detected
Initial Severity	Initial risk assessment
User / Account	Identity associated with the activity
Source IP	Relevant for authentication/network alerts
Command Line	What was executed
Parent Process	What launched the process
Related Alerts	Other events around the same time

Investigation Checklist

- Host: is this the expected endpoint for the activity?
- User: is the account authorized to perform the activity? Is it privileged?
- Process: what process executed? What were its parent and child processes?
- Command: What exactly was executed? Is it encoded, obfuscated, or otherwise unusual?
- Timing: Did the event occur during expected work or maintenance activity?
- Network: is there a relevant source/destination IP, port, or connection?
- Timeline: What happened immediately before and after the alert?
- Correlation: are there other detections involving the same host, user, or time period?
- Context: is there a legitimate explanation for the behavior?

Severity Classification

Severity	Definition	Typical Indicators	Typical Action
Low	Limited security significance or likely benign activity	Expected administration; isolated weak indicator	Close / document
Medium	Suspicious activity requiring investigation	Unusual PowerShell; isolated authentication anomaly	Investigate
High	Strong indication of malicious behavior	Encoded execution; credential attack; suspicious persistence	Escalate when supported
Critical	Confirmed or highly probable compromise with significant impact	Active compromise; successful credential compromise; lateral movement	Immediate escalation

Disposition Categories

Disposition	Meaning	Typical Action
True Positive — Malicious	Evidence supports malicious activity	Escalate
True Positive — Suspicious	Detection is valid and suspicious, but compromise	Continue investigation or escalate

	is not confirmed	
Benign / Expected Activity	Activity is legitimate and expected	Close with justification
False Positive	Detection incorrectly identified legitimate behavior	Close; consider tuning if recurring
Requires Further Investigation	Evidence is insufficient for a confident classification	Collect more evidence or escalate

Detection-Specific Triage Guidance

Detection	Check	Suspicious Indicators	Benign Indicators
Suspicious PowerShell	User, command line, parent/child process, network activity	Execution-policy bypass; unusual parent; download/execute behavior	Authorized administrator script or maintenance
Encoded PowerShell	Command content, user, parent process, network destination	Obfuscation; encoded command; download; credential access; persistence	Known enterprise automation/security tooling
Windows Command Shell	cmd.exe command line, parent process, user	Unexpected cmd.exe; suspicious commands; chained execution	Normal administrative or diagnostic commands
Scheduled Task	Task name, creator, command, path,	Random name; user-writable	Known application task or authorized

	user	script/executable; unusual creator	maintenance
SSH Password Guessing	Source IP, usernames, frequency, subsequent login	Repeated failures; multiple usernames; unexpected source; later success	Known user mistyping credentials or authorized testing

Escalation Triggers by Detection

- PowerShell: suspicious command execution with an unusual parent process, unknown user, or related malicious indicators.
- Encoded PowerShell: encoded/obfuscated execution without a legitimate explanation, especially with network activity or persistence.
- Windows Command Shell: suspicious commands, unusual parent process, or correlation with other detections.
- Scheduled Task: unknown creator, suspicious script/executable, unusual task name/path, or evidence of persistence.
- SSH Password Guessing: repeated attacks, multiple targeted usernames, privileged-account targeting, or a successful login following failures.

Escalation Criteria

- There is evidence of malicious execution.
- Multiple suspicious detections correlate on the same endpoint.
- Credential attacks are repeated or appear successful.
- A persistence mechanism is created or modified suspiciously.
- Encoded or obfuscated commands execute without a legitimate explanation.
- Suspicious activity involves an administrative account.
- There is evidence of lateral movement.
- A critical system may be affected.

- The analyst cannot confidently establish that the activity is legitimate and the potential impact is significant.

Closure Criteria

- The activity is confirmed legitimate.
- The activity is an expected administrative or system operation.
- The detection is determined to be a false positive.
- Investigation finds no supporting indicators of malicious activity.
- The alert has been sufficiently investigated and the findings are documented.

Every closed alert must contain a concise justification. Example: “Closed as benign. The PowerShell command was executed by an authorized administrator during scheduled maintenance. No suspicious child processes, network activity, or related alerts were observed.”

Analyst Triage Record

Each investigated alert should be recorded using the following structure:

Field	Analyst Entry
Alert ID	
Timestamp	
Host / Agent	
Rule ID	
Rule Description	
Initial Severity	
Observed Evidence	1. 2. 3.
Investigation	1. 2.

	3.
Final Severity	
Disposition	
Reasoning	
Action	

Disposition	Meaning
True Positive - Malicious	The alert represents malicious behavior in the investigation context.
True Positive - Suspicious	The behavior is suspicious and requires investigation, but malicious intent is not fully established.
True Positive - Simulated	The detection correctly identified controlled activity generated for testing.
False Positive	The rule fired on legitimate behavior that does not represent the intended threat.
Benign / Expected	The activity is legitimate and expected in the environment.

Completed Triage Log

The following is the consolidated triage log derived from the completed project activities. Where exact timestamps or fields were not necessary to demonstrate the analyst decision, the report keeps the entry concise.

#	Detection	Host	Key Evidence	Disposition
1	Suspicious PowerShell	Windows-Endpoint	PowerShell execution telemetry	True Positive Simulated

2	Encoded PowerShell	Windows-Endpoint	Encoded/obfuscated PowerShell telemetry	True Positive Simulated
3	Windows Command Shell	Windows-Endpoint	cmd.exe command execution; observed net user test	True Positive Simulated
4	Scheduled Task	Windows-Endpoint	schtasks.exe /create /tn myTriageTask	True Positive Simulated
5	Local Account Creation	Windows-Endpoint	net user backdoor backdoor /add	True Positive Simulated
6	SSH Password Guessing	Ubuntu-Endpoint	Repeated sshd authentication failures; custom threshold alert	True Positive Simulated

Conclusion

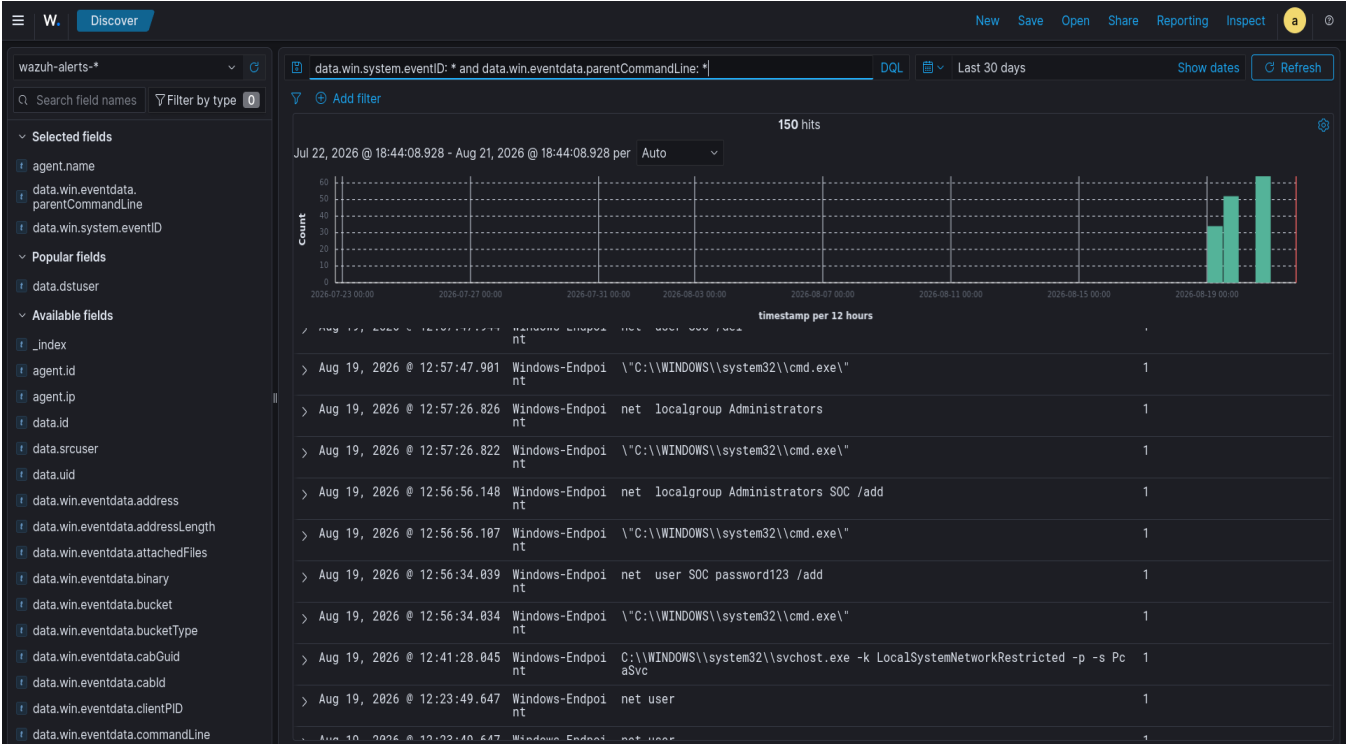
This project provided an end-to-end practical exercise in detection engineering and SOC alert triage. I validated endpoint telemetry, developed custom Wazuh detections, mapped the detection objectives to MITRE ATT&CK, generated controlled security activity, verified the resulting alerts, investigated the events and documented analyst decisions.

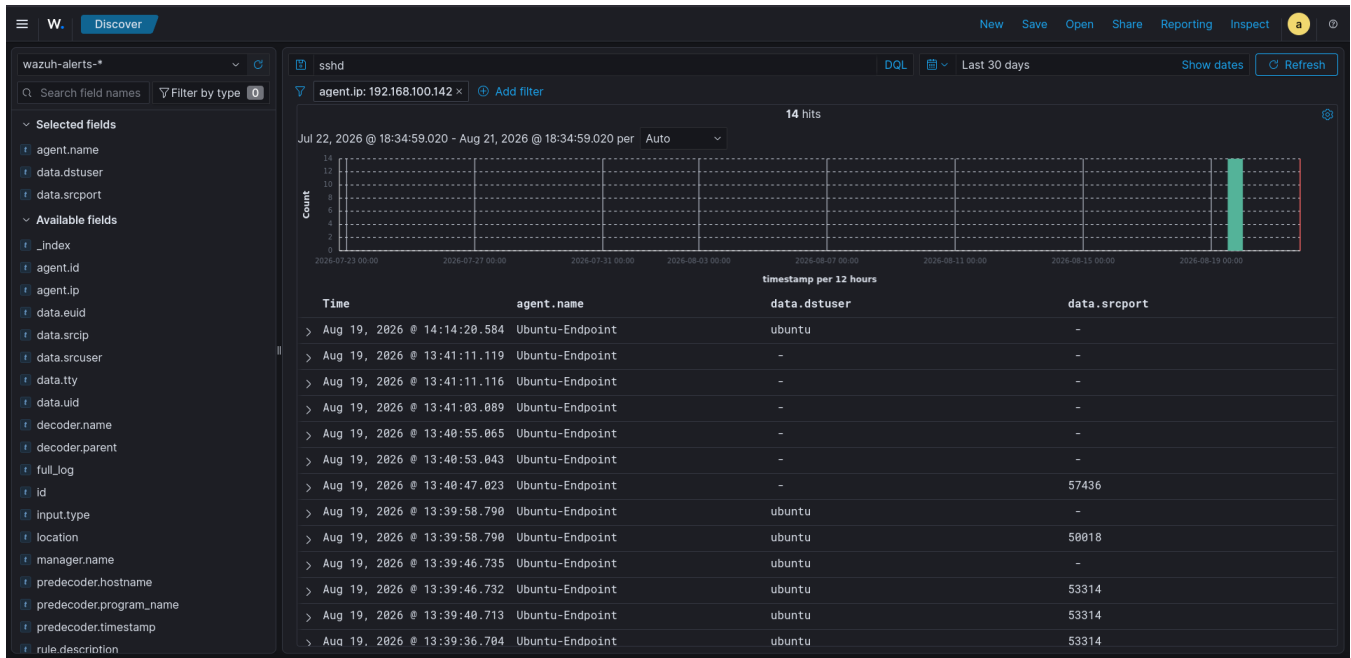
The most important outcome was the transition from simply creating rules to demonstrating how a SOC analyst would use those rules. The investigation stage required

examining command lines, users, endpoints, source IPs, process context and authentication events before assigning a disposition.

The project therefore demonstrates both sides of the detection lifecycle: engineering a detection that can identify a behavior and operating that detection as part of an investigation workflow.

Appendix A - Evidence Placement Guide





```
Administrator: Command Prompt

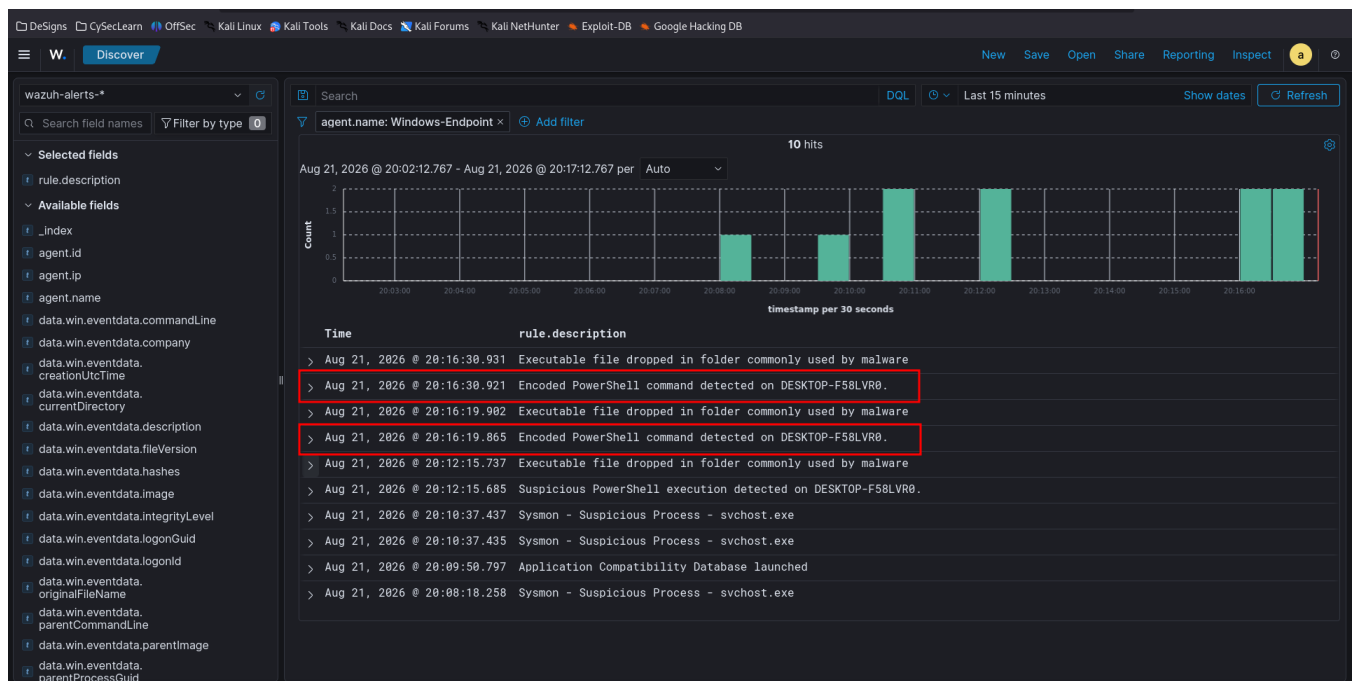
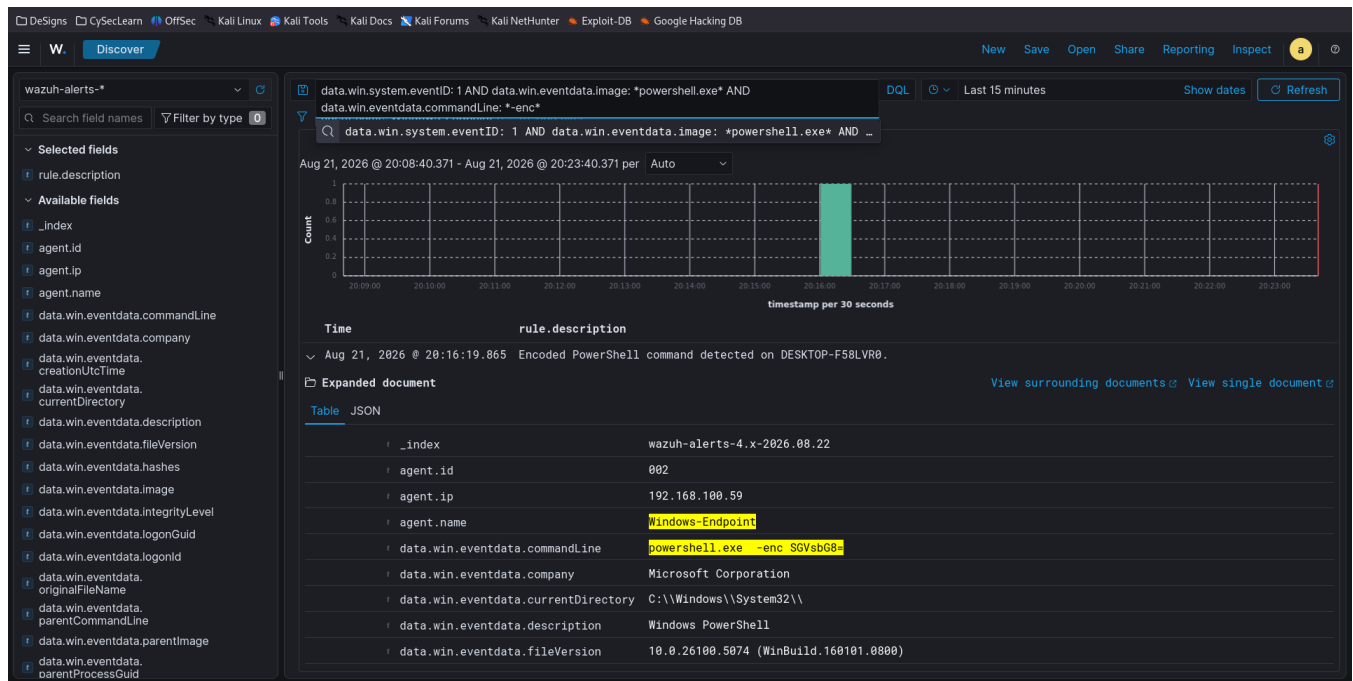
C:\Windows\System32>schtasks /create /tn "DetectionEngineeringTest" /tr "cmd.exe /c echo DetectionTest" /sc once /st 23:59
WARNING: The task name "DetectionEngineeringTest" already exists. Do you want to replace it (Y/N)? Y
SUCCESS: The scheduled task "DetectionEngineeringTest" has successfully been created.

C:\Windows\System32>schtasks /create /tn "DetectionEngineeringTest" /tr "cmd.exe /c echo DetectionTest" /sc once /st 23:59
WARNING: The task name "DetectionEngineeringTest" already exists. Do you want to replace it (Y/N)? ^C

C:\Windows\System32>schtasks /create /tn "DetectionEngineeringTest1" /tr "cmd.exe /c echo DetectionTest1" /sc once /st 23:59
SUCCESS: The scheduled task "DetectionEngineeringTest1" has successfully been created.

C:\Windows\System32>powershell.exe -EncodedCommand SGVsbG8=
效菜 : The term '效菜' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the
spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ 效菜
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (效菜:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

C:\Windows\System32>
```



```
Administrator: Windows PowerShell - cmd.exe net user localgroup - cmd.exe net localgroup Administrators
C:\Windows\System32>
C:\Windows\System32>
C:\Windows\System32>
C:\Windows\System32>
C:\Windows\System32>
C:\Windows\System32>
C:\Windows\System32>
C:\Windows\System32>exit

C:\Windows\System32>cmd.exe net user localgroup
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

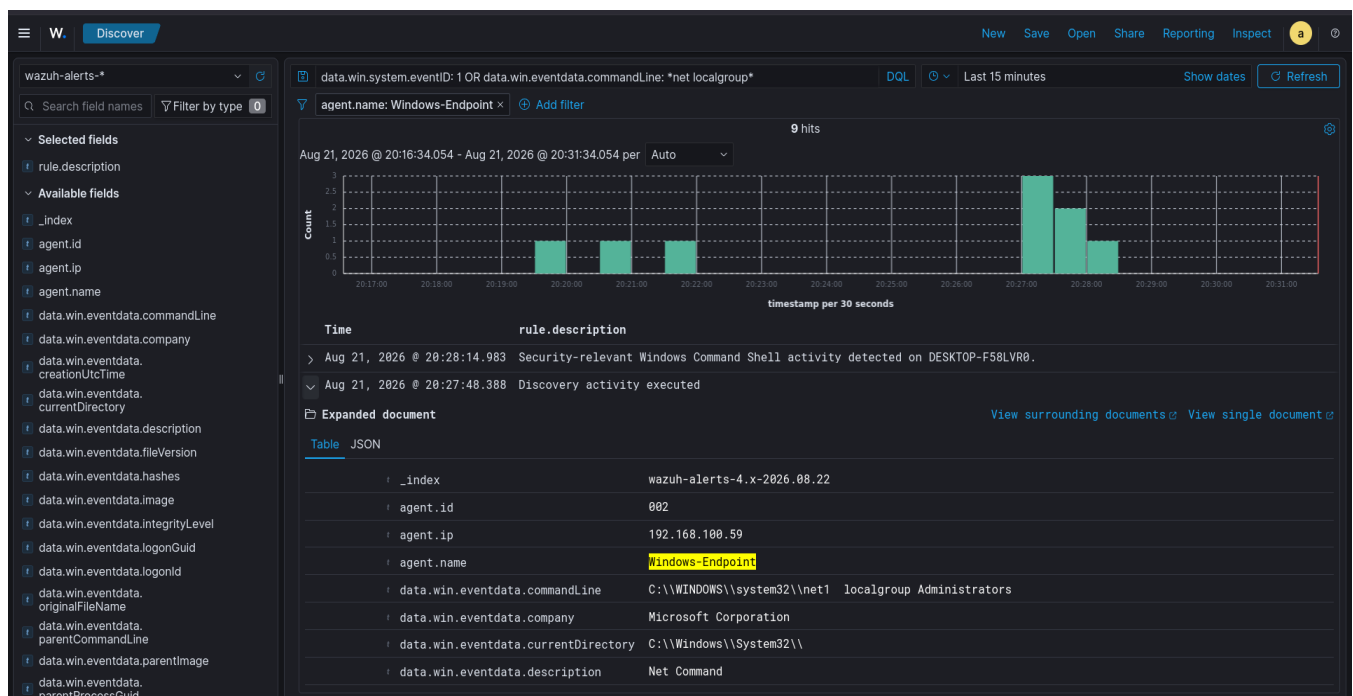
C:\Windows\System32>net localgroup Administrators
Alias name     Administrators
Comment       Administrators have complete and unrestricted access to the computer/domain

Members

-----
Administrator
lucky
The command completed successfully.

C:\Windows\System32>cmd.exe net localgroup Administrators
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

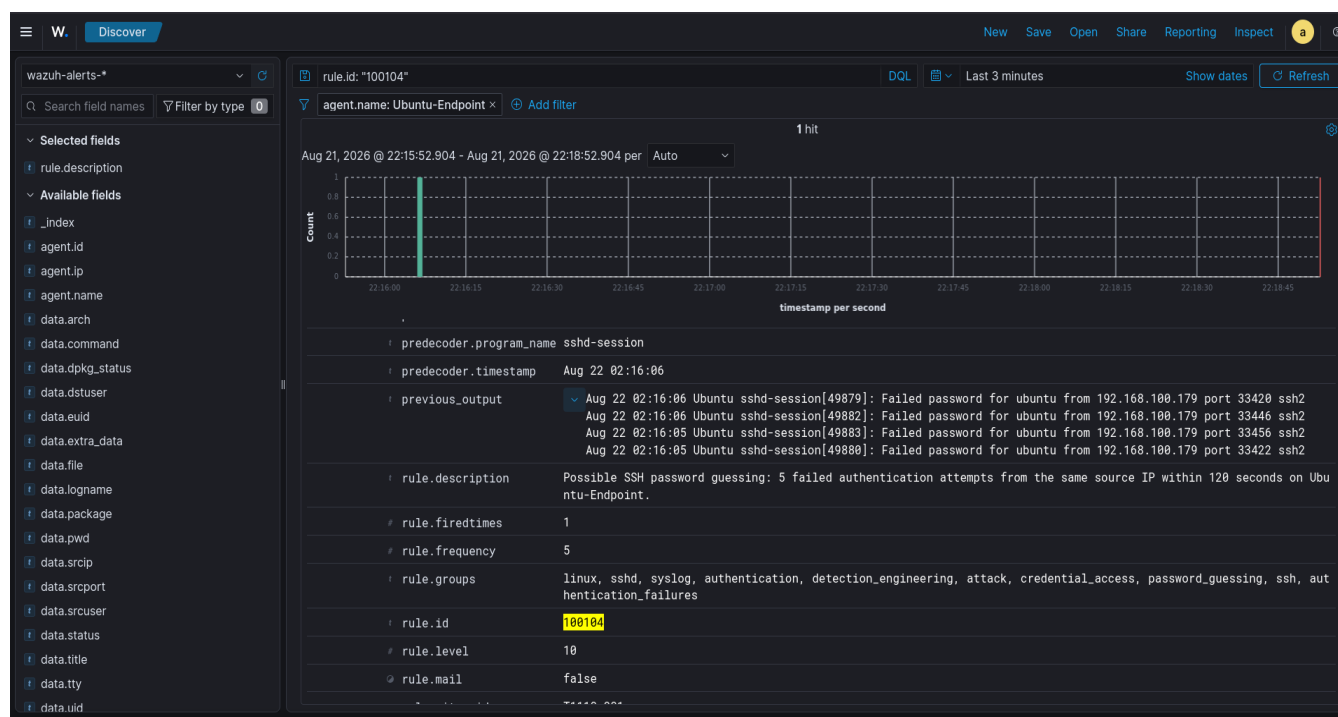
C:\Windows\System32>
```



```
kali@localhost:~$ ssh ubuntu@192.168.100.142
ubuntu@192.168.100.142's password:
Permission denied, please try again.
ubuntu@192.168.100.142's password:
Permission denied, please try again.
ubuntu@192.168.100.142's password:
ubuntu@192.168.100.142: Permission denied (publickey,password).
kali@localhost:~$ ssh ubuntu@192.168.100.142
ubuntu@192.168.100.142's password:
Permission denied, please try again.
ubuntu@192.168.100.142's password:
Permission denied, please try again.
ubuntu@192.168.100.142's password:
ubuntu@192.168.100.142: Permission denied (publickey,password).
kali@localhost:~$
```

Bruteforce command:

```
sudo hydra -l ubuntu -P sshTest.txt -t 6 ssh://192.168.100.142 -vv
```



< local_rules.xml

```
75 <mitre>
76 <id>T1059.003</id>
77 </mitre>
78
79 <group>attack,execution,windows_command_shell,</group>
80 </rule>
81
82 </group>
83
84
85 <group name="windows,sysmon,scheduled_task,persistence,detection_engineering,">
86
87 <rule id="100103" level="11">
88 <if_group>sysmon_event1</if_group>
89
90 <field name="win.eventdata.image" type="pcre2">(?!)\schtasks\.exe$</field>
91
92 <field name="win.eventdata.commandLine" type="pcre2">(?!)(^|[:space:])/ (create|change)([:space:]]|$)</field>
93
94 <description>Scheduled task creation or modification detected on $(win.system.computer).</description>
95
96 <mitre>
97 <id>T1053.005</id>
98 </mitre>
99
100 <group>attack,persistence,scheduled_task,</group>
101 </rule>
102
103 </group>
104
105
106
```

```
</group> →
<group name="linux,sshd,syslog,authentication,detection_engineering,">

<rule id="100104" level="10" frequency="5" timeframe="120" ignore="60">

<if_matched_sid>5760</if_matched_sid>

<same_srcip />

<description>Possible SSH password guessing: 5 failed authentication attempts from the same source IP within 120 seconds on $(hostname).</description>

<mitre>
<id>T1110.001</id>
</mitre>

<group>attack,credential_access,password_guessing,ssh,authentication_failures,</group>

</rule>

</group>
```

Appendix B - Detection Reference

This appendix summarizes the detection set used in the project. Exact XML implementation should be taken from the working rules file submitted with the project; this report does not reproduce the entire ruleset.

Detection	Platform	ATT&CK	Observed / Tested Evidence
Suspicious PowerShell	Windows	T1059.001	PowerShell process execution and command telemetry.
Encoded PowerShell	Windows	T1059.001	Encoded/obfuscated PowerShell test telemetry.
Windows Command Shell	Windows	T1059.003	cmd.exe /c command execution.
Scheduled Task	Windows	T1053.005	"C:\WINDOWS\system32\schtasks.exe" /create /tn myTriageTask
SSH Password Guessing	Ubuntu	T1110.001	Repeated sshd authentication failures; custom rule 100104 observed.
Local Account Creation	Windows	T1136.001	net user backdoor backdoor /add